# 目录

OneAPI Hook Manifest（注入点清单）

一、重要说明

二、7 个注入点

注入点 1：请求入口 — 鉴权 + 预冻结

注入点 2：非流式响应 — 结算

注入点 3：流式处理循环 — 实时余额监控

注入点 4：错误处理 — 退还冻结金额

注入点 5：Failover 循环 — 价格检查

注入点 6：渠道配置 — 上下文窗口

注入点 7（v2.0 新增）：流式 chunk handler — Token 估算

三、注入点之间的数据流

# OneAPI Hook Manifest（注入点清单）

> **版本**：v2.0 **日期**：2026-06-18 **用途**：开发团队在 OneAPI 源码中注入 Hook 的精确位置参考 **变更**：基于 v1.0 修正，从"同构可替换"改为"概念借鉴+重新实现"，补充第 7 个注入点

---

## 一、重要说明

**OneAPI 的 `preConsumeQuota` / `postConsumeQuota` 与 UniRoute 的 Freeze/Settle/Cancel 是概念借鉴关系，不是同构替换。**

| 维度 | OneAPI quota | UniRoute billing |
|---|---|---|
| **数据类型** | Token 数量（int64） | 法币金额（int64，单位：分） |
| **存储** | `users.quota` 字段 | `wallets.balance` + `wallets.frozen` |
| **原子性** | `UPDATE users SET quota = quota - ?` | `UPDATE wallets SET balance = balance - ?, frozen = frozen + ?` |
| **退还** | `ReturnPreConsumedQuota()` | `Cancel()` → `frozen -= amount, balance += amount` |

**结论**：借鉴 OneAPI 的注入点位置和流程控制模式，但计费数据模型完全重新实现。

---

## 二、7 个注入点

### 注入点 1：请求入口 — 鉴权 + 预冻结

**文件**：`controller/relay.go` **函数**：`Relay()` **位置**：请求进入后、转发前

**注入逻辑**：

```
// 1. 提取 API Key
apiKey := c.GetHeader("Authorization")

// 2. 鉴权（替换 OneAPI 原生鉴权）
orgID, keyID, err := auth.ValidateAPIKey(apiKey)
if err != nil {
    return err
}

// 3. 敏感词检查
if filter.ContainsSensitiveWord(reqBody) {
    return NewError(400, "validation_error.sensitive_word_detected")
}

// 4. 预冻结
freezeAmount := billing.CalculateFreeze(reqBody, modelPrices)
if err := billing.Freeze(orgID, freezeAmount, requestID); err != nil {
    return NewError(402, "billing_error.insufficient_balance")
}

c.Set("org_id", orgID)
c.Set("freeze_amount", freezeAmount)
c.Set("request_id", requestID)
```

---

### 注入点 2：非流式响应 — 结算

**文件**：`relay/controller/text.go` **函数**：`RelayTextHelper()` **位置**：收到上游 200 响应后

**注入逻辑**：

```
// 解析 usage
usage := parseUsage(response)
actualCost := calculateCost(usage, modelPrices)

// 结算
billing.Settle(orgID, freezeAmount, actualCost, requestID)

// 写调用日志（异步）
go calllog.Write(requestID, orgID, usage, actualCost, latency)

// 追加 _uniroute 字段
response["_uniroute"] = buildExtension(requestID, actualCost, provider, channel, tt
```

### 注入点 3：流式处理循环 — 实时余额监控

**文件**：`relay/controller/text.go`（流式处理部分） **函数**：流式 SSE 处理循环 **位置**：每个 chunk 处理后

**注入逻辑**：

```
// StreamChunkHandler 接口
type StreamChunkHandler interface {
    OnChunk(chunk []byte, tokenCount int) error
    OnComplete(usage *Usage) error
    OnError(err error) error
}

// 实现
func (h *billingChunkHandler) OnChunk(chunk []byte, tokenCount int) error {
    h.localTokenCount += tokenCount

    // 每 100 token 检查余额
    if h.localTokenCount % 100 == 0 {
        balance := redis.Get("balance:" + h.orgID)
        if balance < 0 {
            // 余额穿底：强制断开上游
            h.upstreamCancel()
            return ErrBalanceExhausted
        }
    }

    // 转发给客户端
    return h.clientWriter.Write(chunk)
}
```

### 注入点 4：错误处理 — 退还冻结金额

**文件**：`relay/controller/text.go` **函数**：错误处理分支 **位置**：请求失败时

**注入逻辑**：

```
if err != nil {
    // 退还冻结金额
    billing.Cancel(orgID, freezeAmount, requestID)

    // 写调用日志（失败）
    go calllog.WriteError(requestID, orgID, err, latency)

    // 熔断器计数
    circuitBreaker.RecordFailure(channelID)
}
```

---

## 注入点 5：Failover 循环 — 价格检查

**文件**：`controller/relay.go` **函数**：Failover 循环 **位置**：切换渠道前

**注入逻辑**：

```
for _, channel := range channels {
    // 检查熔断器状态
    if circuitBreaker.IsOpen(channel.ID) {
        if !circuitBreaker.ShouldTryHalfOpen(channel.ID) {
            continue
        }
    }

    // 选择下一个可用 Key
    key := channel.NextAvailableKey()
    if key == nil {
        continue
    }

    // 转发请求
    resp, err := relay(channel, key, req)
    if err == nil {
        circuitBreaker.RecordSuccess(channel.ID)
        return resp
    }

    // 记录失败
    circuitBreaker.RecordFailure(channel.ID)

    // 判断是否可重试
    if !isRetryableError(err) {
        break
    }
}
```

---

### 注入点 6：渠道配置 — 上下文窗口

**文件**：`relay/meta/` **函数**：渠道配置解析 **位置**：渠道加载时

**注入逻辑**：

```
// 从 channels 表读取配置
channel := &Channel{
    ID:              row.ID,
    Name:            row.Name,
    Provider:        row.Provider,
    BaseURL:         row.BaseURL,
    Models:          parseModels(row.Models),
    ModelMapping:    parseJSON(row.ModelMapping),
    MaxContext:       row.MaxContext,
    DefaultMaxOutput: row.DefaultMaxOutput,
    RPM:             row.RPM,
    TPM:             row.TPM,
    Priority:        row.Priority,
    Status:          row.Status,
}
```

---

### 注入点 7（v2.0 新增）：流式 chunk handler — Token 估算

**文件**：`relay/controller/stream.go`（新文件） **函数**：`StreamChunkHandler.OnComplete()` **位置**：流结束时

**注入逻辑**：

```
func (h *billingChunkHandler) OnComplete(usage *Usage) error {
    var actualCost int64
    var usageSource string

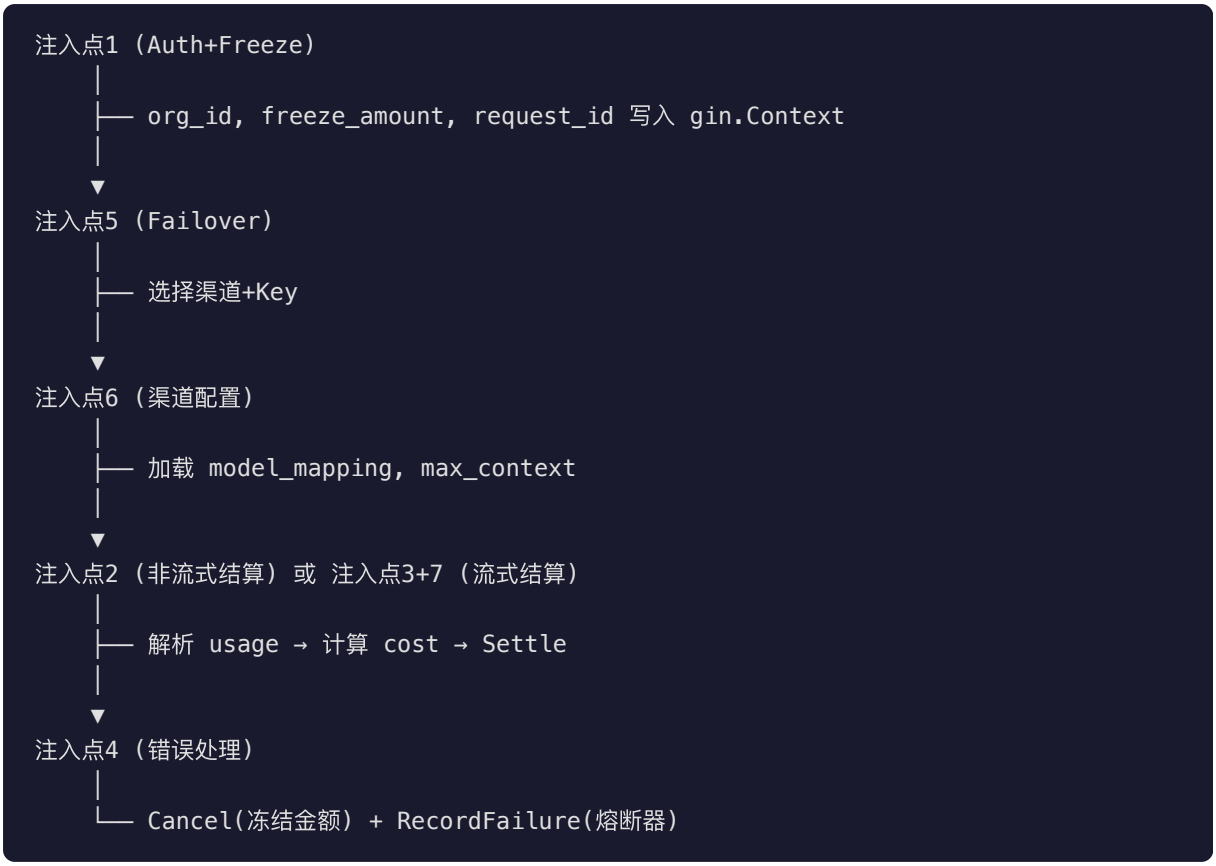
    if usage != nil {
        // 厂商返回了 usage
        actualCost = calculateCost(usage, h.modelPrices)
        usageSource = "upstream"
    } else {
        // 厂商未返回 usage，本地估算
        estimatedTokens := estimateTokens(h.localContentBuffer)
        usage = &Usage{
            PromptTokens:    h.promptTokens,
            CompletionTokens: estimatedTokens,
        }
        actualCost = calculateCost(usage, h.modelPrices)
        usageSource = "estimated"
    }

    // 结算
    billing.Settle(h.orgID, h.freezeAmount, actualCost, h.requestID)

    // 写调用日志
    go calllog.Write(h.requestID, h.orgID, usage, actualCost, h.latency, usageSourc

    return nil
}
```

## 三、注入点之间的数据流


注入点1 (Auth+Freeze)
org_id, freeze_amount, request_id 写入 gin.Context
注入点5 (Failover)
选择渠道+Key
注入点6 (渠道配置)
加载 model_mapping, max_context
注入点2 (非流式结算) 或 注入点3+7 (流式结算)
解析 usage → 计算 cost → Settle
注入点4 (错误处理)
Cancel(冻结金额) + RecordFailure(熔断器)